Guía de estudio · Segundo parcial: estructuras de programación

El segundo parcial es **práctico**: vas a escribir funciones. Casi todos los problemas se resuelven combinando unas pocas **estructuras** que se repiten una y otra vez. Esta guía te enseña a **reconocer cómo se ve cada una** para que, al leer un problema, sepas qué esqueleto escribir.

Todos los ejemplos están en ejemplo_estructuras.py, ordenados **de menos a más complejos**. Ejecútalo y compara lo que imprime con el comentario `#` de cada `print`. Cada ejemplo regresa un valor con `return`. Los ejemplos usan situaciones de todos los días: tu playlist, tus partidas, el fútbol, los conciertos, tus ahorros.

> **Convención de nombres:** toda variable o parámetro que guarda una **lista** empieza con `lista_` (`lista_playlist`, `lista_resultados`, `lista_partidos`…). Así, con sólo leer el nombre sabes que puedes recorrerla con `for`, pedirle `len()` o un elemento con `[i]`.

Cada sección trae:

- **Esqueleto:** la forma general. Las palabras como `condicion` o `lista_elementos` son las piezas que tú cambias.
- **Ejemplo:** un caso mínimo que funciona.
- **Cómo reconocerla:** las pistas en el enunciado de un problema.
- ⚠️ **Error típico:** lo que más se equivoca en los exámenes.

Índice

1. Función
2. Variables y tipos
3. `if` sin `else`
4. `if` / `else`
5. `if` / `elif` / `else`
6. `if` anidados
7. `and`, `or`, `not`
8. `for` por elementos + acumulador
9. Tipos de acumuladores
10. `for` con índices
11. Izquierda → derecha y derecha → izquierda
12. Al mismo tiempo: dos extremos y dos listas
13. Recorrido circular con `%`
14. `while`
15. `for` dentro de `for`
16. Recursividad
17. Lista de diccionarios
18. Tabla resumen: ¿qué estructura uso?

1. Función

Una función recibe datos (**parámetros**), hace algo con ellos y **regresa** un resultado.

Esqueleto

```
def nombre_funcion(par_1, par_2, par_n):
    # La lógica de la función va aquí
    return 0


print(nombre_funcion(1, 2, 3))  # 0
```

Puntos clave

- `def`, el nombre, los paréntesis y los **dos puntos** `:` al final.
- Todo lo que está **dentro** de la función lleva sangría (4 espacios).
- `return` **regresa** el valor a quien llamó la función y **termina** la función. `print` sólo lo muestra en pantalla.
- Definir la función no la ejecuta: se ejecuta cuando la **llamas**, `nombre_funcion(1, 2, 3)`, con un valor para cada parámetro.

⚠️ **Error típico:** usar `print` en lugar de `return`. La función "se ve" bien en la terminal, pero regresa `None`.

2. Variables y tipos

Ejemplo: el perfil de un jugador. Cada dato se guarda en una variable de su tipo, y al final todos se juntan en un diccionario, que es lo que regresa la función.

```
def crear_perfil():
    usuario = "ana_gamer"  # Texto (str)
    nivel = 42  # Entero (int)
    horas_jugadas = 310.5  # Decimal (float)
    es_premium = True  # Booleano (bool)
    lista_juegos = ["Minecraft", "Fortnite", "Mario Kart"]  # Lista (list)
    perfil = {  # Diccionario (dict): junta todos los datos del perfil
        "usuario": usuario,
        "nivel": nivel,
        "horas_jugadas": horas_jugadas,
        "es_premium": es_premium,
        "juegos": lista_juegos,
    }
    return perfil


perfil_ana = crear_perfil()
print(perfil_ana["usuario"])  # ana_gamer
print(perfil_ana["juegos"])  # ['Minecraft', 'Fortnite', 'Mario Kart']
```

Puntos clave

- Una sola `=` **asigna**: guarda un valor en una variable. Dos `==` **comparan**.
- `42` y `"42"` son distintos: uno es número y el otro texto.
- En una lista, los elementos se piden por **posición**: `lista_juegos[0]` es `"Minecraft"`. En un diccionario se piden por **llave**: `perfil_ana["usuario"]` es `"ana_gamer"`.
- Un diccionario puede guardar valores de cualquier tipo, incluso una lista (`"juegos": lista_juegos`).
- Una función puede no tener parámetros: los paréntesis van vacíos, `crear_perfil()`.

3. `if` sin `else`

La decisión más sencilla: hay un bloque para cuando la condición **se cumple**, y ningún bloque para cuando no. Si no se cumple, el programa simplemente continúa con la siguiente línea después del `if`. Se usa de dos maneras muy distintas:

Uso 1: cambiar un valor por defecto

La variable empieza con un **valor por defecto**, y el `if` sólo la cambia cuando hace falta.

Esqueleto

```
resultado = valor_por_defecto
if condicion:
    resultado = otro_valor
return resultado
```

Ejemplo: ¿qué canción suena primero en la playlist?

```
def primera_cancion(lista_playlist):
    cancion = None  # valor por defecto
    if len(lista_playlist) > 0:
        cancion = lista_playlist[0]  # sólo cambia si se cumple la condición
    return cancion


print(primera_cancion(["Tusa", "La Bachata", "Ojitos Lindos"]))  # Tusa
```

Uso 2: validar un edge case

El `if` revisa un **edge case** (una playlist vacía, un divisor en cero, un dato inválido) y, si se cumple, **termina la función** con `return`. Aquí que la condición se cumpla no es "algo extra": es la salida de emergencia. El resto de la función, el caso normal, sólo se ejecuta si la validación **no** se cumplió.

Esqueleto

```
if edge_case:
    return valor_para_el_edge_case  # la función termina aquí
# caso normal: sólo se llega aquí si el edge case no se cumplió
return resultado_normal
```

Ejemplo: la misma función, escrita como validación.

```python
def primera_cancion(lista_playlist):
    if len(lista_playlist) == 0:  # edge case: la playlist está vacía
        return None  # la función termina aquí
    return lista_playlist[0]  # caso normal: sólo se llega si hay canciones


print(primera_cancion([]))  # None
print(primera_cancion(["Tusa", "La Bachata", "Ojitos Lindos"]))  # Tusa
```

Puntos clave

- No hace falta `else`: como el `return` ya terminó la función, la línea de abajo sólo se alcanza cuando la validación no se cumplió.
- Las validaciones van **al inicio** de la función, antes de usar los datos. Aquí protege a `lista_playlist[0]`, que con una playlist vacía lanzaría `IndexError`.
- Una función puede tener varias validaciones seguidas, una por cada edge case.

Cómo reconocerla:

- Uso 1: "si pasa X, cambia/agrega/cuenta…", y no dice qué hacer en el otro caso.
- Uso 2: "si la lista está vacía regresa…", "si el dato no es válido regresa…".

⚠️ **Error típico:** en el uso 2, olvidar el `return` dentro del `if`. La validación "detecta" el edge case pero la función sigue de largo y truena en la línea de abajo.

4. `if` / `else`

Dos caminos: si la condición se cumple se ejecuta un bloque; si no, **el otro**. A diferencia del `if` solo, aquí **siempre** se ejecuta uno de los dos.

Esqueleto

```python
if condicion:
    # se cumple
else:
    # no se cumple
```

Ejemplo

```python
def es_mayor_edad(edad):
    if edad >= 18:
        return True
    else:
        return False


print(es_mayor_edad(20))  # True
```

La misma función, con la mayoría de edad como parámetro

El `18` está "escrito a mano" dentro de la función. Pero la mayoría de edad no es igual en todos los países (en Estados Unidos, para algunas cosas, es 21). Si la convertimos en un **parámetro**, la misma función sirve para cualquier caso:

```python
def es_mayor_edad(edad, mayoria_edad):
    if edad >= mayoria_edad:
        return True
    else:
        return False


print(es_mayor_edad(20, 21))  # False
```

Cómo reconocerla: "si … regresa X, si no regresa Y".

⚠️ **Errores típicos:** olvidar los `:` o la sangría. Escribir `>` cuando el enunciado dice "mayor **o igual**": con `>`, `es_mayor_edad(18)` regresaría `False`.

5. `if` / `elif` / `else`

Tres o más caminos. Python revisa las condiciones **en orden, de arriba a abajo**, y ejecuta **sólo el primer bloque** cuya condición se cumple. El `else` final atrapa todo lo demás.

Esqueleto

```
if condicion_1:
    # camino 1
elif condicion_2:
    # camino 2
elif condicion_3:
    # camino 3
else:
    # ninguna se cumplió
```

Ejemplo: el plan del viernes depende de cuántos amigos se apuntan.

```
def plan_de_viernes(lista_amigos):
    if len(lista_amigos) == 0:
        return "maratón de series en casa"
    elif len(lista_amigos) == 1:
        return "ir al cine"
    elif len(lista_amigos) <= 5:
        return "noche de videojuegos"
    else:
        return "fiesta"


print(plan_de_viernes(["Ana", "Beto", "Caro"]))  # noche de videojuegos
```

Cómo reconocerla: rangos o categorías ("bronce si tienes menos de 100 puntos, plata si tienes menos de 500, oro si…").

⚠️ **Error típico:** el **orden**. Si la condición más general va primero, las de abajo nunca se alcanzan. En el ejemplo, si `len(lista_amigos) <= 5` fuera la primera condición, con cero amigos el plan sería `"noche de videojuegos"`. Regla: **lo más específico va primero.**

6. `if` anidados

Un `if` **dentro** de otro: la segunda pregunta sólo se hace si la primera se cumplió.

Esqueleto

```
if condicion_1:
    if condicion_2:
        # se cumplen las dos
    else:
        # se cumple la 1 pero no la 2
else:
    # no se cumple la 1
```

Ejemplo: la entrada a un concierto. Primero se revisa el boleto; **sólo si lo tienes**, se revisa la edad para darte la pulsera.

```
def acceso_concierto(lista_con_boleto, nombre, edad):
    if nombre in lista_con_boleto:
        if edad >= 18:
            return "pasa con pulsera de mayor de edad"
        else:
            return "pasa sin pulsera"
    else:
        return "no tienes boleto"


print(acceso_concierto(["Ana", "Beto", "Caro"], "Beto", 17))  # pasa sin pulsera
```

Puntos clave

- Cada nivel lleva **4 espacios más** de sangría. Cada `else` pertenece al `if` que está **a su misma altura**: el último `else` es la respuesta a "¿tienes boleto?", no a "¿eres mayor de edad?".
- `nombre in lista_con_boleto` es `True` si el nombre está en la lista.
- Un `else:` que sólo contiene un `if` se puede escribir como `elif`.

Cómo reconocerla: "si pasa X, entonces revisa si también pasa Y" (por ejemplo los años bisiestos: divisible entre 4 → ¿y entre 100? → ¿y entre 400?).

7. `and`, `or`, `not`

Combinan condiciones en una sola.

Operador	Es `True` cuando...	Ejemplo
`a and b`	**las dos** son `True`	`edad >= 18 and nacionalidad == "mexicana"`
`a or b`	**al menos una** es `True`	`dia == "miercoles" or es_estudiante`
`not a`	`a` es `False` (lo voltea)	`not (cancion in lista_playlist)`

Ejemplos

Para votar en México hay que tener 18 años **y** ser de nacionalidad mexicana:

```
def puede_votar(edad, nacionalidad):
    return edad >= 18 and nacionalidad == "mexicana"


print(puede_votar(18, "mexicana"))  # True
```

En el cine hay descuento los miércoles **o** con credencial de estudiante:

```
def descuento_en_cine(dia, es_estudiante):
    return dia == "miercoles" or es_estudiante


print(descuento_en_cine("sabado", True))  # True
```

¿La canción **no** está en tu playlist?

```
def falta_en_playlist(lista_playlist, cancion):
    return not (cancion in lista_playlist)


print(falta_en_playlist(["Tusa", "La Bachata"], "Ojitos Lindos"))  # True
```

Puntos clave

- Una comparación **ya es** `True` o `False`, así que se puede regresar directamente: `return edad >= 18` en vez de `if edad >= 18: return True else: return False`.
- `and` se evalúa antes que `or`. **Si los mezclas, usa paréntesis:** `es_estudiante and (dia == "martes" or dia == "miercoles")`.
- `and` deja de revisar en cuanto encuentra un `False`. En `len(lista_playlist) > 0 and lista_playlist[0] == "Tusa"`, si la playlist está vacía nunca se llega a `lista_playlist[0]` y no hay error.
- Dos `if` anidados en los que **las dos** condiciones deben cumplirse son lo mismo que un `and`.

⚠️ **Error típico:** `dia == "sabado" or "domingo"`. Cada lado del `or` debe ser una comparación completa: `dia == "sabado" or dia == "domingo"`.

8. `for` por elementos + acumulador

El `for` visita **cada elemento** de la lista, uno por uno. El **acumulador** es una variable que va guardando el resultado parcial en cada vuelta.

Esqueleto (los 4 pasos del acumulador)

```
acumulador = valor_inicial  # 1. antes del ciclo
for elemento in lista_elementos:  # 2. recorrer
    acumulador = acumulador + elemento  # 3. actualizar dentro del ciclo
return acumulador  # 4. regresar DESPUÉS del ciclo
```

Ejemplo: ¿cuántos minutos dura tu playlist?

```
def duracion_playlist(lista_minutos):
    total = 0  # 1. valor inicial del acumulador
    for minutos in lista_minutos:  # 2. recorrer
        total = total + minutos  # 3. actualizar el acumulador
    return total  # 4. regresar, FUERA del ciclo


print(duracion_playlist([3, 4, 2, 5]))  # 14
```

Prueba de escritorio (hazla en papel para cualquier ciclo):

vuelta	minutos	total
antes	—	0
1	3	3
2	4	7
3	2	9
4	5	14

Cómo reconocerla: "suma", "cuenta", "total", "cuántos", "promedio", "el mejor".

⚠️ **Errores típicos:**

- Poner `total = 0` **dentro** del ciclo: se reinicia en cada vuelta.
- Poner el `return` **dentro** del ciclo (con sangría de más): la función termina en la primera vuelta.

9. Tipos de acumuladores

Todos tienen los mismos 4 pasos. Lo que cambia es el **valor inicial** y **cómo se actualiza**.

Acumulador	Valor inicial	Actualización	Ejemplo en el `.py`
Suma	`0`	`total = total + x`	`duracion_playlist`
Contador	`0`	`contador = contador + 1` (dentro de un `if`)	`contar_victorias`
Producto	`1`	`producto = producto * x`	`combinaciones_outfit`
Mínimo / máximo	el primer elemento	`if x < mejor: mejor = x`	`mejor_vuelta`
Lista nueva	`[]`	`lista_nueva.append(x)`	`precios_al_alcance`
Varios a la vez	uno por cada dato	cada uno con su regla	`promedio_goles`

Contador: ¿cuántos partidos ganaste? (`"G"` ganado, `"P"` perdido, `"E"` empate)

```python
def contar_victorias(lista_resultados):
    victorias = 0
    for resultado in lista_resultados:
        if resultado == "G":
            victorias = victorias + 1
    return victorias


print(contar_victorias(["G", "P", "G", "E", "G"]))  # 3
```

Producto: con 3 playeras, 2 pantalones y 4 pares de tenis, ¿cuántos outfits distintos puedes armar?

```python
def combinaciones_outfit(lista_opciones):
    combinaciones = 1
    for opciones in lista_opciones:
        combinaciones = combinaciones * opciones
    return combinaciones


print(combinaciones_outfit([3, 2, 4]))  # 24
```

Mínimo: tu mejor vuelta en Mario Kart es la que tardó **menos** segundos.

```python
def mejor_vuelta(lista_segundos):
    mejor = lista_segundos[0]
    for segundos in lista_segundos:
        if segundos < mejor:
            mejor = segundos
    return mejor


print(mejor_vuelta([92, 87, 95]))  # 87
```

Lista nueva: de los precios de unos tenis, ¿cuáles te alcanzan con tu presupuesto?

```python
def precios_al_alcance(lista_precios, presupuesto):
    lista_alcanzables = []
    for precio in lista_precios:
        if precio <= presupuesto:
            lista_alcanzables.append(precio)
    return lista_alcanzables


print(precios_al_alcance([1200, 499, 899, 1500], 900))  # [499, 899]
```

Varios a la vez: el promedio de goles por partido necesita la suma **y** cuántos partidos hubo.

```python
def promedio_goles(lista_goles):
    total = 0
    partidos = 0
    for goles in lista_goles:
        total = total + goles
        partidos = partidos + 1
    return total / partidos


print(promedio_goles([2, 0, 3, 1]))  # 1.5
```

⚠️ **Errores típicos:**

- Producto que empieza en `0`: todo da 0 (cero outfits).
- Mínimo que empieza en `0`: ninguna vuelta tarda menos de 0 segundos, así que regresa 0, que ni siquiera está en la lista. Empieza con el primer elemento, `lista_segundos[0]`. (Para el máximo pasa lo mismo con números negativos.)
- Contador sin `if`: cuenta **todos** los partidos, no sólo los ganados.

10. `for` con índices

Se usa cuando importa **la posición** del elemento y no sólo su valor: para regresar dónde está algo, comparar un elemento con su vecino o recorrer dos listas a la vez.

Esqueleto

```
for i in range(0, len(lista_elementos), 1):  # i = 0, 1, 2, ..., len - 1
    # lista_elementos[i] es el elemento en la posición i
```

Ejemplo (con `return` temprano): ¿en qué posición de la playlist está una canción?

```
def posicion_de_cancion(lista_playlist, cancion):
    for i in range(0, len(lista_playlist), 1):  # i = 0, 1, 2, ..., len - 1
        if lista_playlist[i] == cancion:
            return i  # termina la función en cuanto la encuentra
    return -1  # sólo llega aquí si revisó todo y no la encontró


print(posicion_de_cancion(["Tusa", "La Bachata", "Ojitos Lindos"], "La Bachata"))  # 1
```

Puntos clave

- **Cómo leer `range(0, len(lista_playlist), 1)`:** empieza en `0`, para **antes de** `len(lista_playlist)` (el fin no se incluye) y avanza de **1 en 1**.
- `for cancion in lista_playlist` te da **valores**; `for i in range(0, len(lista_playlist), 1)` te da **posiciones**. Con la posición siempre puedes obtener el valor (`lista_playlist[i]`), pero no al revés.
- Las posiciones empiezan en **0**: la segunda canción está en la posición 1. La última está en `len(lista_playlist) - 1`; `lista_playlist[len(lista_playlist)]` **no existe** (`IndexError`).
- Un `return` dentro del ciclo **detiene la búsqueda** en cuanto se encuentra la respuesta. El `return` "de fracaso" (`-1`) va **después** del ciclo.

⚠️ **Error típico:** poner `else: return -1` dentro del ciclo. La función se rinde después de revisar sólo la primera canción.

11. Izquierda → derecha y derecha → izquierda

La dirección la decide `range(inicio, fin, paso)`. `fin` **no se incluye**.

Dirección	range	Posiciones que visita (lista de 5)
Izquierda → derecha	`range(0, len(lista_resultados), 1)`	0, 1, 2, 3, 4
Derecha → izquierda	`range(len(lista_resultados) - 1, -1, -1)`	4, 3, 2, 1, 0

Ejemplos: en los resultados de la temporada, ¿en qué jornada fue tu **primera** victoria y en cuál la **última**? La misma búsqueda en las dos direcciones da respuestas distintas. Como la jornada 1 está en la posición

0, la función regresa `i + 1`.

```
def jornada_primera_victoria(lista_resultados):
    for i in range(0, len(lista_resultados), 1):  # 0, 1, 2, ..., len - 1
        if lista_resultados[i] == "G":
            return i + 1  # las posiciones empiezan en 0, las jornadas en 1
    return -1  # no hubo ninguna victoria


print(jornada_primera_victoria(["P", "G", "E", "G", "P"]))  # 2


def jornada_ultima_victoria(lista_resultados):
    for i in range(len(lista_resultados) - 1, -1, -1):  # len - 1, ..., 2, 1, 0
        if lista_resultados[i] == "G":
            return i + 1  # las posiciones empiezan en 0, las jornadas en 1
    return -1  # no hubo ninguna victoria


print(jornada_ultima_victoria(["P", "G", "E", "G", "P"]))  # 4
```

Cómo leer `range(0, len(lista_resultados), 1)`: empieza en la **primera** posición, para **antes de** `len(lista_resultados)` y avanza de **1 en 1**.

Cómo leer `range(len(lista_resultados) - 1, -1, -1)`: empieza en la **última** posición, para **antes de** `-1` (o sea, incluye el 0) y avanza de **-1 en -1**.

Cómo reconocerla: "el primero que…" → izquierda a derecha. "El último que…", "el más reciente", "al revés" → derecha a izquierda.

⚠️ **Error típico:** `range(len(lista_resultados), 0, -1)`. Empieza en una posición que no existe (`IndexError`) y nunca visita la posición 0.

12. Al mismo tiempo: dos extremos y dos listas

a) Dos índices que se acercan desde los extremos

Un índice empieza a la izquierda, el otro a la derecha, y se mueven **en la misma vuelta** hasta encontrarse. Como no se sabe de antemano cuántas vueltas habrá, se usa `while`.

Esqueleto

```
izquierda = 0
derecha = len(lista_elementos) - 1
while izquierda < derecha:
    # comparar lista_elementos[izquierda] con lista_elementos[derecha]
    izquierda = izquierda + 1
    derecha = derecha - 1
```

Ejemplo: ¿tu gamertag se lee igual al derecho y al revés? Un texto se recorre igual que una lista: `gamertag[0]` es su primera letra y `len(gamertag)` cuántas letras tiene.

```
def es_palindromo(gamertag):
    izquierda = 0
    derecha = len(gamertag) - 1
    while izquierda < derecha:
        if gamertag[izquierda] != gamertag[derecha]:
            return False
        izquierda = izquierda + 1
        derecha = derecha - 1
    return True


print(es_palindromo("otto"))  # True
```

vuelta	izquierda	derecha	compara
1	0	3	`o` con `o` ✓
2	1	2	`t` con `t` ✓
—	2	1	se detiene (`2 < 1` es `False`)

b) Dos listas con el mismo índice

Esqueleto

```
for i in range(0, len(lista_a), 1):
    # lista_a[i] y lista_b[i] son "pareja"
```

Ejemplo: la quiniela. Comparas tu pronóstico de cada partido con el resultado real (`"L"` gana local, `"E"` empate, `"V"` gana visitante).

```python
def aciertos_quiniela(lista_pronosticos, lista_resultados):
    aciertos = 0
    for i in range(0, len(lista_resultados), 1):
        if lista_pronosticos[i] == lista_resultados[i]:
            aciertos = aciertos + 1
    return aciertos


print(aciertos_quiniela(["L", "E", "V", "L"], ["L", "V", "V", "L"]))  # 3
```

Cómo reconocerla: "palíndromo", "comparar el primero con el último" → dos extremos. "Comparar partido por partido", "respuesta por respuesta" → dos listas.

⚠️ **Error típico:** en dos extremos, olvidar mover **uno** de los índices: el ciclo nunca termina.

13. Recorrido circular con `%`

`x % n` siempre da un número entre `0` y `n - 1`. Por eso sirve para que una posición **dé la vuelta**: después del último elemento sigue el primero (como las horas del reloj o los días de la semana).

Esqueleto

```python
posicion = (posicion + 1) % len(lista_elementos)
```

Ejemplo: en el coche, cada amigo elige una canción por turno. Si empieza quien está en la posición 2 (Caro), ¿quién elige cada una de las siguientes 4 canciones?

```python
def turnos_dj(lista_amigos, posicion_inicial, cantidad_canciones):
    lista_turnos = []
    posicion = posicion_inicial
    for turno in range(0, cantidad_canciones, 1):
        lista_turnos.append(lista_amigos[posicion])
        posicion = (posicion + 1) % len(lista_amigos)
    return lista_turnos


print(turnos_dj(["Ana", "Beto", "Caro"], 2, 4))  # ['Caro', 'Ana', 'Beto', 'Caro']
```

Puntos clave

- Después de la posición 2 (`"Caro"`, la última), `(2 + 1) % 3` da `0` (`"Ana"`).
- También funciona hacia atrás: `(0 - 1) % 3` da `2`.

Cómo reconocerla: "por turnos", "da la vuelta", "vuelve a empezar", "aparece del otro lado" (Pacman), horas, minutos, días de la semana.

14. `while`

Repite **mientras** la condición sea `True`. A diferencia del `for`, **tú** controlas el inicio, la condición y el avance.

Esqueleto (las 3 partes)

```
i = 0  # 1. inicio
while i < len(lista_elementos):  # 2. condición para seguir
    # trabajar con lista_elementos[i]
    i = i + 1  # 3. avance
```

Ejemplo 1: el mismo recorrido que un `for` (la duración de la playlist de la sección 8)

```
def duracion_con_while(lista_minutos):
    total = 0
    i = 0  # 1. inicio
    while i < len(lista_minutos):  # 2. condición para seguir
        total = total + lista_minutos[i]
        i = i + 1  # 3. avance (sin esta línea, el ciclo es infinito)
    return total


print(duracion_con_while([3, 4, 2, 5]))  # 14
```

Ejemplo 2: detenerse cuando se cumple algo. Ahorras cada semana para un boleto de concierto de $500. ¿Cuántas semanas necesitas? No sabemos cuántas vueltas habrá: el ciclo para en cuanto juntas el dinero. Si se acaban las semanas y no alcanzó, la función regresa -1 (el edge case, validado con un `if` sin `else` como en la sección 3).

```python
def semanas_para_boleto(lista_ahorros, precio):
    total = 0
    semanas = 0
    while semanas < len(lista_ahorros) and total < precio:
        total = total + lista_ahorros[semanas]
        semanas = semanas + 1
    if total < precio:  # edge case: se acabaron las semanas y no alcanzó
        return -1
    return semanas


print(semanas_para_boleto([200, 150, 300, 250], 500))  # 3
print(semanas_para_boleto([100, 50], 500))  # -1
```

vuelta	semanas antes	total después	¿sigue? (total < 500)
1	0	200	sí
2	1	350	sí
3	2	650	no → regresa `semanas = 3`

¿`for` o `while`?

Usa `for` cuando...	Usa `while` cuando...
vas a recorrer **toda** la lista	te detienes cuando **se cumple algo**
sabes cuántas vueltas habrá	no sabes cuántas vueltas habrá
—	mueves dos índices de forma distinta (dos extremos, búsqueda binaria)

⚠️ **Errores típicos:**

- Olvidar el avance (`semanas = semanas + 1`): **ciclo infinito**. La plataforma lo corta a los 10 s.
- En una condición con `and`, revisar `semanas < len(lista_ahorros)` **primero**: protege a `lista_ahorros[semanas]` de pedir una semana que no existe.

15. `for` dentro de `for`

Por **cada vuelta** del ciclo de afuera, el ciclo de adentro da **todas** sus vueltas.

Esqueleto

```
for a in lista_afuera:
    for b in lista_adentro:
        # se ejecuta len(afuera) × len(adentro) veces
```

Ejemplo 1: lista de listas. Un torneo tiene varias jornadas y cada jornada tiene varios partidos. ¿Cuántos goles hubo en todo el torneo?

```
def goles_del_torneo(lista_jornadas):
    total = 0
    for lista_partidos in lista_jornadas:  # ciclo de afuera: cada jornada
        for goles in lista_partidos:  # ciclo de adentro: cada partido de esa jornada
            total = total + goles
    return total


print(goles_del_torneo([[2, 1, 3], [0, 4, 1]]))  # 11
```

Ejemplo 2: comparar cada elemento con los demás. Quieres comprar **dos** videojuegos y tienes $800. ¿Cuántas parejas distintas de juegos te alcanzan?

```
def parejas_de_juegos(lista_precios, presupuesto):
    parejas = 0
    for i in range(0, len(lista_precios), 1):
        for j in range(i + 1, len(lista_precios), 1):  # j empieza después de i
            if lista_precios[i] + lista_precios[j] <= presupuesto:
                parejas = parejas + 1
    return parejas


print(parejas_de_juegos([300, 500, 200, 700], 800))  # 3
```

Las parejas que alcanzan son 300 + 500, 300 + 200 y 500 + 200. Empezar `j` en `i + 1` evita emparejar un juego consigo mismo y contar dos veces la misma pareja (300 + 500 y 500 + 300).

Cómo reconocerla: "tabla", "jornadas y partidos", "lista de listas", "cada pareja", "comparar cada uno con todos", "¿hay repetidos?".

Big O: con n elementos, dos ciclos anidados dan del orden de n × n vueltas: **O(n²)**.

⚠️ **Error típico:** usar el mismo nombre de variable en los dos ciclos (`i` y `i`): el ciclo de adentro pisa al de afuera.

16. Recursividad

Una función **recursiva se llama a sí misma** con un problema **más pequeño**, hasta llegar a un caso tan simple que se responde directo.

Esqueleto (las 2 partes obligatorias)

```
def funcion_recursiva(problema):
    if es_el_caso_base:  # 1. caso base: se responde sin llamarse otra vez
        return respuesta_directa
    else:  # 2. caso recursivo: se llama con un problema MÁS PEQUEÑO
        return combinar(parte, funcion_recursiva(problema_mas_pequeno))
```

Ejemplo 1: recortando la lista. La duración de la playlist es "la primera canción + la duración del resto". `lista_minutos[1:]` es "la lista sin su primer elemento".

```
def duracion_recursiva(lista_minutos):
    if len(lista_minutos) == 0:  # caso base: el problema más pequeño
        return 0
    else:  # caso recursivo: la primera canción + la duración del resto
        return lista_minutos[0] + duracion_recursiva(lista_minutos[1:])


print(duracion_recursiva([3, 4, 2, 5]))  # 14
```

Cómo se resuelve. Las llamadas bajan hasta el caso base y los resultados suben:

```
duracion_recursiva([3, 4, 2, 5])
= 3 + duracion_recursiva([4, 2, 5])
= 3 + 4 + duracion_recursiva([2, 5])
= 3 + 4 + 2 + duracion_recursiva([5])
= 3 + 4 + 2 + 5 + duracion_recursiva([])
= 3 + 4 + 2 + 5 + 0                    ← caso base: playlist vacía
= 14
```

Ejemplo 2: con un índice (avanza la posición en lugar de recortar la lista)

```python
def duracion_recursiva_indice(lista_minutos, i):
    if i == len(lista_minutos):  # caso base: ya no quedan canciones
        return 0
    else:
        return lista_minutos[i] + duracion_recursiva_indice(lista_minutos, i + 1)


print(duracion_recursiva_indice([3, 4, 2, 5], 0))  # 14
```

Cómo reconocerla: el enunciado dice "recursivo", o el problema se define en términos de sí mismo: "el factorial de n es n × el factorial de n - 1".

⚠️ **Errores típicos:**

- Sin caso base, o con un caso base que nunca se alcanza: `RecursionError`.
- Llamarse con el **mismo** problema (`duracion_recursiva(lista_minutos)`) en lugar de uno más pequeño: tampoco termina nunca.
- Olvidar el `return` antes de la llamada recursiva: el resultado se pierde y regresa `None`.

17. Lista de diccionarios

Cada elemento de la lista es un diccionario. Es la forma típica de guardar "registros", como jugadores, canciones o productos.

Esqueleto

```python
for registro in lista_registros:
    registro["llave"]  # el valor de esa llave para este registro
```

Ejemplo: ¿quiénes del equipo metieron 5 goles o más en la temporada?

```python
def goleadores(lista_jugadores):
    lista_goleadores = []
    for jugador in lista_jugadores:
        if jugador["goles"] >= 5:
            lista_goleadores.append(jugador["nombre"])
    return lista_goleadores


lista_equipo = [
    {"nombre": "Ana", "goles": 7},
    {"nombre": "Beto", "goles": 2},
    {"nombre": "Caro", "goles": 5},
]
print(goleadores(lista_equipo))  # ['Ana', 'Caro']
```

Es un `for` por elementos con un acumulador de lista (secciones 8 y 9). La única novedad es que cada `jugador` es un diccionario y sus datos se piden por **llave**.

⚠️ **Error típico:** escribir la llave con otro nombre o sin comillas (`jugador[goles]`). La llave debe escribirse exactamente igual que en el diccionario y entre comillas.

18. Tabla resumen: ¿qué estructura uso?

Si el problema dice…	Estructura	Sección
"si pasa X, cambia/agrega…" (sin otro caso)	`if` sin `else`	3
"si la lista está vacía / el dato no es válido, regresa…"	`if` de validación con `return`	3
"si … si no …"	`if` / `else`	4
rangos, categorías, 3 o más casos	`if` / `elif` / `else`	5
"si X, y además si Y…"	`if` anidados **o** `and`	6, 7
"todos deben cumplir", "a la vez"	`and`	7
"cualquiera de", "alguno de"	`or`	7
"no es", "ninguno"	`not`	7
suma, total, promedio	`for` + acumulador (empieza en 0)	8, 9
"cuántos cumplen…"	`for` + `if` + contador	9
el mayor / el menor / el mejor	acumulador que empieza con el primer elemento	9
"regresa una lista con los que…"	acumulador `[]` + `append`	9
"en qué posición", "el índice"	`for i in range(0, len(lista_elementos), 1)`	10
"el primero que…"	izquierda → derecha + `return` temprano	10, 11
"el último que…", "al revés"	derecha → izquierda	11
palíndromo, "desde los dos extremos"	dos índices + `while`	12
comparar dos listas posición por posición	un índice para las dos listas	12
turnos, "da la vuelta", horas, días	`% len(lista_elementos)`	13
"hasta que…", "mientras…"	`while`	14
tabla, lista de listas, cada pareja, repetidos	`for` dentro de `for`	15
"de forma recursiva"	caso base + caso recursivo	16
registros con nombre, goles, precio…	lista de diccionarios	17

Antes de entregar cualquier ejercicio, revisa:

1. ¿Mi función tiene `return`, y está **fuera** del ciclo, a menos que quiera terminar antes?
2. ¿Mi acumulador empieza **antes** del ciclo y con el valor correcto (0, 1, `[]`, el primer elemento)?
3. ¿Probé con una lista **vacía**, con **un solo** elemento y con el elemento buscado al **inicio** y al **final**?
4. ¿Uso `>=` o `>` según lo que dice el enunciado?
5. Si uso `while`: ¿algo cambia en cada vuelta para que el ciclo termine?